EX NO: 8b	SIMULATION AND ANALYSIS OF QPSK MODULATION USING CONSTELLATION DIAGRAMS IN MATLAB
DATE:10/08/26	

Objective

i) To generate QPSK symbols from a binary data sequence using quadrature phase-shift keying principles.

ii) To plot and analyze the QPSK signal constellation in the In-Phase (I) and Quadrature (Q) plane.

iii) To investigate the effect of noise on constellation points and evaluate the resulting degradation in symbol detection performance.

Matlab code

```
clc;
clear;
close all;

%% Objective 1: Generate QPSK Symbols
N = 1000;                    % Number of bits
data = randi([0 1], N, 1);   % Random binary sequence

% Ensure even number of bits
if mod(N,2)~=0
data = [data; 0];
N = N + 1;
end

% Group bits into pairs
bit_pairs = reshape(data,2,[])';

% QPSK Mapping

qpsk_symbols = zeros(size(bit_pairs,1),1);
for k = 1:size(bit_pairs,1)
if isequal(bit_pairs(k,:),[0 0])
qpsk_symbols(k) = (1+1j)/sqrt(2);
        elseif isequal(bit_pairs(k,:),[0 1])
qpsk_symbols(k) = (-1+1j)/sqrt(2);
```

```
        elseif isequal(bit_pairs(k,:),[1 1])
qpsk_symbols(k) = (-1-1j)/sqrt(2);
        elseif isequal(bit_pairs(k,:),[1 0])
qpsk_symbols(k) = (1-1j)/sqrt(2);
        end
end

%% Objective 2: Plot QPSK Constellation

scatter(real(qpsk_symbols),imag(qpsk_symbols), 30);
xlim([-1 1])
ylim([-1 1])
hold on;

xline(0,'k','LineWidth',1.5);
yline(0,'k','LineWidth',1.5);

grid on;
axis equal;

xlabel('In-Phase (I)');
ylabel('Quadrature (Q)');
title('Ideal QPSK Constellation');

%% Objective 3: Add Noise and Evaluate Performance
SNR = 5;

rx_signal = awgn(qpsk_symbols,SNR,'measured');

figure;
scatter(real(rx_signal),imag(rx_signal), 15);
hold on;

xline(0,'k','LineWidth',1.5);
yline(0,'k','LineWidth',1.5);

grid on;
axis equal;
xlabel('In-Phase (I)');
ylabel('Quadrature (Q)');
title(['Noisy QPSK Constellation (SNR = ',num2str(SNR),' dB)']);

%% Symbol Detection
detected_bits = zeros(length(rx_signal)*2,1);
```

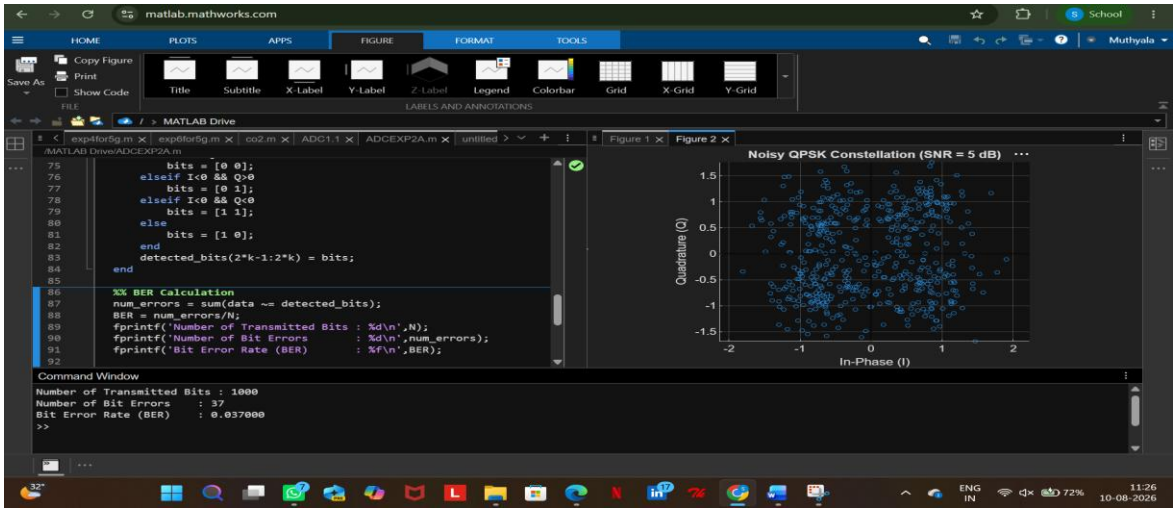
```
for k = 1:length(rx_signal)
I = real(rx_signal(k));
Q = imag(rx_signal(k));
if I>0 && Q>0
bits = [0 0];
elseif I<0 && Q>0
bits = [0 1];
elseif I<0 && Q<0
bits = [1 1];
else
bits = [1 0];
end
detected_bits(2*k-1:2*k) = bits;
end

%% BER Calculation
num_errors = sum(data ~= detected_bits);
BER = num_errors/N;
fprintf('Number of Transmitted Bits : %d\n',N);
fprintf('Number of Bit Errors       : %d\n',num_errors);
fprintf('Bit Error Rate (BER)       : %f\n',BER);
```

Objective (i): Generate QPSK Symbols from a Binary Data Sequence Using Quadrature Phase-Shift Keying Principles

Procedure:

1. Initialize the MATLAB environment and define the number of bits.
2. Generate a random binary sequence consisting of 0s and 1s.
3. Group the binary sequence into pairs of bits.
4. Apply QPSK modulation by mapping each bit pair to a unique constellation point.
5. Generate the corresponding complex QPSK symbols.
6. Verify that four distinct symbol values are obtained.
7. Store the generated symbols for constellation analysis.

OUTPUT GRAPH



Result

QPSK symbols were successfully generated from the binary data sequence using quadrature phase-shift keying modulation.